



文件操作 - 课堂练习解答



存储TODOLIST

- **新手任务：记录程序运行情况**

为了在计算机系统中对时间进行统一的表示和处理。**时间戳**是一种方便的方式来记录事件发生的时间、计算时间间隔、进行排序、进行数据同步等操作。它在各种应用程序中被广泛使用，包括操作系统、数据库、网站、移动应用程序等。



存储TODO LIST

- **新手任务：记录程序运行情况**

- 程序启动时将时间写到start.log (创建新文件，如果有旧文件存在，追加到旧文件)

- 程序结束时将时间**追加**写到start.log

例如运行2次后应存在4条时间记录



邀请2位同学讲讲思路



知识点

- 打开文件的方式
- 追加：a
- 不追加（不存在时新建，存在时覆盖）：w
- 不追加（新建，但存在时报错）：x

值	说明
r	读模式
w	写模式，不存在时创建,文件存在时 截取覆盖原有文件！！
x	写模式，先创建文件然后打开写;文 件存在时抛异常FileExistsError
a	附加模式，不存在时会创建，存在 时会附加到文件尾部
t	文本模式
b	二进制模式
+	读写模式，较少使用！！可使用r+ w+ x+ a+，注意w会清空原有文件的 内容



尝试运行多次

```
import time
```

```
fobj = open('app.log', 'w')
```

```
fobj.write('START:'+time.ctime(time.time()))+'\n')
```

```
fobj.write('do something\n')
```

```
fobj.write('END:'+time.ctime(time.time()))+'\n')
```

```
fobj.close()
```



尝试运行多次

```
import time
```

```
fobj = open('app.log', 'a')
```

```
fobj.write('START:'+time.ctime(time.time()))+'\n')
```

```
fobj.write('do something\n')
```

```
fobj.write('END:'+time.ctime(time.time()))+'\n')
```

```
fobj.close()
```



题目答案

```
import time
```

```
fobj = open('app.log', 'a')
```

```
fobj.write('START:'+time.ctime(time.time()))+'\n')
```

```
fobj.close()
```

```
# DO SOMETHING
```

```
fobj = open('app.log', 'a')
```

```
fobj.write('END:'+time.ctime(time.time()))+'\n')
```

```
fobj.close()
```



为了看得更清楚

```
import time
```

```
fobj = open('app.log', 'w')
```

```
fobj.write('START:'+time.ctime(time.time()))+'\n')
```

```
fobj.close()
```

```
i = input() # 等待输入时打开app.log观察效果
```

```
fobj = open('app.log', 'a')
```

```
fobj.write('END:'+time.ctime(time.time()))+'\n')
```

```
fobj.close()
```



存储TODOLIST

- **第一关：读取/保存数据**

现阶段程序的数据保存在计算机的内存中，内存是一种**易失性**存储器件，即一旦程序终止（断电、程序异常、操作系统重启等）就会丢失其中的数据。因此，如果想要长期保存数据，就需要使用一种**非易失性**存储器件，如磁盘、固态硬盘等。



存储TODOLIST

- **第一关：读取/保存数据**

- 把用户的TODOLIST保存到磁盘文件（请思考使用何种格式）
 - 程序启动时将读取文件中记录的TODOLIST数据
- 程序正常终止时（QUIT）将把内存中的TODOLIST数据存储在文件（覆盖旧文件）

这里无需用户选择文件名



邀请2位同学讲讲思路



注意点1

- 数据存储的类型
- 几次作业后的要求：所有输入的待完成事项SOMETHING1 ,
DEADLINE1 , LEVEL1 , TIME1
- SOMETHING2 , DEADLINE2 , LEVEL2 , TIME2
- 还有其他的风格吗？



注意点1

- 数据重复出现（序列）
- 数据成组记录（事件，日期，时间等）
- 可能存在空数据



注意点2

- 程序启动时读取
- 程序终止时存储
- 这一对操作相反
- `f.readlines()` vs `f.writelines(lines)`
- `f.readline()` vs `f.write(text)`

文件对象的常用操作

文件打开后会维护一个文件指针位置，给出了下一次的读写操作时所对应的位置。以a模式打开时，指针为文件的当前结束位置，否则为0

	方法	说明
读	f.read([size=-1])	读取最大size个字符(字节)或EOF，size为-1时表示文件中所有字符(字节)
	f.readline()	读取字符直到遇到换行符或者EOF，如果读时马上遇到EOF，则返回空字符串。 注意在 读时遇到\n时，\n也包括在内
	f.readlines()	读取直到EOF，返回一个列表，每个元素为读到的每行内容 (包括行尾换行符\n)
	next(f)	以读方式打开的文件是一个 迭代器 ，下一个元素为文件中下一行的内容 (包括\n) ，直到EOF为止
写	f.write((text))	将字符串(字节串)text写入文件，返回写入的字符(字节)个数
	f.writelines(lines)	把lines中的每个元素逐个写入文件，要求list的元素必须为字符串，返回None
其他	f.flush()	缓冲区的内容同步到存储设备
	f.close()	关闭文件，释放资源，写模式下还同步缓冲区内容到存储设备
	f.truncate([size])	文件截取后长度变为size，缺省为当前文件指针位置(文件打开时初始为0)，返回文件的当前长度
	f.tell()	访问打开文件的当前指针位置
	f.seek(offset, whence=0)	移动文件指针位置到基准位置的offset，缺省whence=0，表示从头部开始，whence=1,表示当前指针位置, whence=2表示文件结尾开始



先搞定存储数据

```
fobj = open('app.db', 'w')
```

```
mylist = [('事件1','2019-01-01','12:00'),('事件2','2020-01-01','12:00'),('事件3','2021-01-01','12:00'),('事件4','2022-01-01','12:00')]
```

```
for t in mylist:
```

```
    str = ""
```

```
    for s in t:
```

```
        str = str + s
```

```
    fobj.write(str+'\n')
```

```
fobj.close()
```



```
fobj = open('app.db', 'w')
```

```
mylist = [('事件1','2019-01-01','12:00'),('事件2','2020-01-01','12:00'),('事件3','2021-01-01','12:00'),('事件4','2022-01-01','12:00')]
```

```
for t in mylist:
```

```
    str = ""
```

```
    for s in t:
```

```
        str = str + s + ','
```

```
    fobj.write(str+'\n')
```

```
fobj.close()
```



```
fobj = open('app.csv', 'w')
```

```
mylist = [('事件1','2019-01-01','12:00'),('事件2','2020-01-01','12:00'),('事件3','2021-01-01','12:00'),('事件4','2022-01-01','12:00')]
```

```
for t in mylist:
```

```
    str = ""
```

```
    for s in t:
```

```
        str = str + s + ','
```

```
    fobj.write(str+'\n')
```

```
fobj.close()
```



再搞定读取数据

```
fobj = open('app.csv', 'r')
```

```
newlist = f.readlines()
```

```
print(newlist)
```

```
#mylist = [('事件1','2019-01-01','12:00'),('事件2','2020-01-01','12:00'),('事件3','2021-01-01','12:00'),('事件4','2022-01-01','12:00')]
```

```
fobj.close()
```



再搞定读取数据

```
fobj = open('app.csv', 'r')
newlist = fobj.readlines()
mylist = []
for s in newlist:
    sList = s.split(",")
    t = tuple(sList)
    mylist.append(t)
print(mylist)
#mylist = [('事件1','2019-01-01','12:00'),('事件2','2020-01-01','12:00'),('事件3','2021-01-01','12:00'),('事件4','2022-01-01','12:00')]
fobj.close()
```



```
fobj = open('app.csv', 'r')
newlist = fobj.readlines()
mylist = []
for s in newlist:
    sList = s.split(",")
    sList.pop()
    t = tuple(sList)
    mylist.append(t)
print(mylist)
#mylist = [('事件1','2019-01-01','12:00'),('事件2','2020-01-01','12:00'),('事件3','2021-01-01','12:00'),('事件4','2022-01-01','12:00')]
fobj.close()
```



更多思考

- 如果没有用Tuple，而是使用了多个不同的列表存储事件、日期、时间
- 如果没有用Tuple，而是只用了一个列表，列表以事项数量的倍数存储数据，例如[item1,level1,deadline1,date1,item2,level2...]
- CSV的逗号分隔是否影响数据
- 难度提升：baidu一下python处理Excel文件
(<https://zhuanlan.zhihu.com/p/156824693>)

存储TODOLIST



- **第二关：指定文件名**

为了支持多个用户同时使用，每位用户需要指定自己的文件名，但可能衍生出文件找不到、磁盘满、权限不足等问题

存储TODOLIST



- **第二关：指定文件名**

- 程序启动时询问用户使用的文件名
 - 如果文件不存在，直接启动，退出时存储数据到指定的文件中
 - 如果存在，打开文件加载数据
 - 需考虑异常处理（打不开，找不到等情况）

知识点



- input()
- 异常处理



异常处理：try

```
fname = input('Input File Name:')
```

```
fobj = None
```

```
try:
```

```
    fobj = open(fname, "r")
```

```
except IOError:
```

```
    print("no file with that name")
```

```
else:
```

```
    newlist = fobj.readlines()
```

```
    print(newlist)
```

```
finally:
```

```
    if fobj:
```

```
        fobj.close()
```

异常处理：with



怎么做？

存储TODO LIST



- **第三关：帮用户记录文件名**

并不是每位用户都熟悉文件结构，所以我们常见的程序把文件操作隐藏起来，我们只需要告诉程序是谁在使用，由程序帮用户创建、打开文件，这里就产生了如何记录的问题



存储TODOLIST

- **第三关：帮用户记录文件名**

- 用一个辅助文件帮用户记录文件名

(**格式请自己思考**)

- 程序启动时首先询问用户名，从name.dat中检查文件信息
 - 如果**没有记录**过，就帮用户生成他的记录文件
 - 如果**记录过**（**辅助文件中包含用户的记录**），就打开对应的记录文件
- **需考虑异常处理（打不开，找不到等情况）**



邀请2位同学讲讲思路



处理思路

- 参考前面内容，组织一个数据结构，可以容易查找{Name:File}
- 是否可以用元组？
 - (Name,File)
- 是否可以用列表？
 - List[1]='Name',保存为1.txt
- 或者直接用名字保存？
 - Zhang -> Zhang.dat



```
def save_dict_to_file(dic):
```

```
    f = open('dict.txt','w')
```

```
    f.write(str(dic))
```

```
    f.close()
```

```
def load_dict_from_file():
```

```
    f = open('dict.txt','r')
```

```
    data=f.read()
```

```
    f.close()
```

```
    return eval(data)
```



引入其他包

```
import json
my_dict = { 'Apple': 4, 'Banana': 2, 'Orange': 6, 'Grapes': 11 }
tf = open("myDictionary.json", "w")
json.dump(my_dict,tf)
tf.close()
```

```
import json
tf = open("myDictionary.json", "r")
new_dict = json.load(tf)
print(new_dict)
```

存储TODOLIST



- **第四关：记录用户的密码**

我们已经能够保存好文件，也能支持多个用户使用，但是数据是用户的隐私，我们需要用密码保护起来

存储TODOLIST



- **第四关：记录用户的密码**

- 检查用户记录文件

- 如果用户不存在，生成他的记录文件，再记录密码到他的文件中
 - 如果存在用户记录，就询问密码
 - 如果密码正确，从对应的记录文件加载信息
 - 如果密码不正确，拒绝打开



邀请2位同学讲讲思路



处理思路

- 组织一个数据结构，可以容易处理即可
 - {Password:"XXX",data:"XXX"}
 - 第一行为密码，其后为数据
 - 任意合理的方案
- 有没有可能分多个文件？
 - user.data记录数据，user.pass记录密码
 - 确保不要移位就可以

存储TODOLIST



Top 30 Most Used Passwords in the World			
1	123456	11	abc123
2	password	12	1234
3	123456789	13	password1
4	12345	14	iloveyou
5	12345678	15	1q2w3e4r
6	qwerty	16	000000
7	1234567	17	qwerty123
8	111111	18	zaq12wsx
9	1234567890	19	dragon
10	123123	20	sunshine
		21	princess
		22	letmein
		23	654321
		24	monkey
		25	27653
		26	1qaz2wsx
		27	123321
		28	qwertyuiop
		29	superman
		30	asdfghjkl

- 第五关：密码复杂度（可选）
- 不接受等级为弱或更低的密码

等级	规则
强	大写字母+小写字母+数字+特殊字符（ !@#\$%^&* ）
中	字母+数字，字母+特殊字符，数字+特殊字符
弱	纯数字，纯字母，纯特殊字符
不符合要求	低于8位



邀请2位同学讲讲思路



处理思路

- 大写字母、小写字母、数字、特殊字符

- 正则表达式可以轻松完成判断

- 用字符串的相关函数也可以完成判断

字符串类型判断: `isalnum()` `isalpha()` `isdecimal()` `isdigit()`
`isidentifier()` `islower()` `isupper()` `isnumeric()`
`isprintable()` `isspace()` `istitle()`

- 密码等级

- 计分机制

- flag机制配合条件判断



纯正则表达式示例（不做要求）

- $^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[!@#\$%^&*])[A-Za-z\d!@#\$%^&*]{8,}$$ （强密码）
 - $(?=.*[A-Z])$ ：正则表达式必须包含至少一个大写字母。
 - $(?=.*[a-z])$ ：正则表达式必须包含至少一个小写字母。
 - $(?=.*\d)$ ：正则表达式必须包含至少一个数字。
 - $(?=.*[!@#\$%^&*])$ ：正则表达式必须包含至少一个特殊字符。
 - $[A-Za-z\d!@#\$%^&*]{8,}$ ：匹配包含大写字母、小写字母、数字、特殊字符的字符串，且长度至少为8个字符。



处理思路

- 额外说明：以明文方式存在安全隐患，其他人可以从数据文件中访问密码信息，需要对密码进行进一步处理（如使用hash()方法等进行处理，[参考网址](#)，此处仅为课程练习需求，正式开发程序切勿使用这样的方式。